

TeeShirtBazz - System Design Document

1. Project Overview

TeeShirtBazz is a full-stack e-commerce platform focused exclusively on selling and customizing t-shirts. It includes authentication, cart, orders, payments, returns, chatbot, and deployment using modern DevOps practices.

2. Tech Stack

- 1 Frontend: React
- 2 Backend: Spring Boot (Java)
- 3 Database: MySQL
- 4 ORM: Spring Data JPA (Hibernate)
- 5 Authentication: JWT
- 6 Cloud: AWS (EC2, RDS, S3)
- 7 CI/CD: GitHub Actions
- 8 Containerization: Docker

3. Core Features

- 1 User Registration & Login
- 2 Product browsing and filtering
- 3 Cart management
- 4 Order placement with transaction safety
- 5 Payment simulation (UPI, Card, COD)
- 6 Order tracking
- 7 Return request system
- 8 Admin product management
- 9 Custom T-shirt builder
- 10 FAQ Chatbot

4. Architecture

Frontend (React) communicates with Backend (Spring Boot) via REST APIs. Backend interacts with MySQL database. Deployment will be done using Docker and AWS services.

5. Database Design (Entities)

- 1 User: id, name, email, password, role
- 2 Product: id, name, category, price, color, size, stock
- 3 Cart & CartItem
- 4 Order & OrderItem
- 5 Payment: method, status, transaction_id
- 6 ReturnRequest: reason, status
- 7 CustomOrder: size, color, logo_url

6. API Endpoints

- 1 POST /auth/register
- 2 POST /auth/login
- 3 GET /products
- 4 POST /cart/add
- 5 POST /orders
- 6 POST /payments
- 7 POST /returns
- 8 POST /custom-orders
- 9 POST /chat

7. Transaction Flow

Order placement is transactional: validate cart, check stock, deduct stock, create order, process payment. Rollback on failure using @Transactional.

8. Return Workflow

User requests return -> Admin reviews -> Approve/Reject -> Process completion.

9. Custom T-Shirt Feature

Users can submit custom design requests including size, color, and logo. Admin reviews and converts into orders.

10. Deployment

- 1 Docker containers for backend and frontend
- 2 AWS EC2 for hosting backend
- 3 AWS RDS for MySQL
- 4 AWS S3 for image storage

11. CI/CD Pipeline

GitHub Actions pipeline: Push -> Build -> Test -> Dockerize -> Deploy to AWS.

12. Differentiators

- 1 Custom T-shirt builder
- 2 Return workflow system
- 3 Focused niche (only t-shirts)
- 4 Simple AI chatbot integration